RENTCAM

Cameroon Rental Marketplace

FULL SYSTEM ANALYSIS &
ARCHITECTURE DESIGN DOCUMENT

Version	1.0 — Initial Release
Date	May 2026
Stack	JavaScript (Next.js / React Native) + PostgreSQL
Status	Pre-Development — Architecture Phase

Table of Contents

Table of Contents 1
1. Product Vision & Scope 2
1.1 Vision Statement 3
1.2 Mission 3
1.3 Product Goals 3
1.4 Scope — What RentCam IS 3
1.5 Scope — What RentCam is NOT 3
1.6 Platform Surfaces 3
2. Functional Requirements 4
2.1 User Roles & Permissions 5
2.2 Feature Breakdown by Module 5
Module 1 — Authentication & Identity 5
Module 2 — Property Listings 5
Module 3 — Search & Discovery 6
Module 4 — Inquiries & Communication 6
Module 5 — Lease Management 7
Module 6 — Payments 7
Module 7 — Reviews & Trust 8
Module 8 — Admin & Operations 8
3. Non-Functional Requirements 9
4. System Architecture 10
4.1 Architecture Overview 11
4.2 High-Level Architecture Diagram 11
4.3 Technology Stack — Detailed 11
4.4 API Architecture 12
4.4.1 API Design Principles 12
4.4.2 Response Envelope 13
4.4.3 Core API Endpoints 13
5. Database Design 14
5.1 Database Principles 15
5.2 Entity Relationship Overview 15
5.3 Full Schema Definition 15
Table: users 15
Table: properties 16
Table: listing_photos 17
Table: inquiries 17
Table: leases 17
Table: payments 18
Table: reviews 18

Table: otp_tokens 19
Additional Supporting Tables 19
6. Application Layer Design 19
6.1 Monorepo Structure 20
6.2 API Server Structure (packages/api) 20
6.3 Next.js Web App Structure (apps/web) 20
6.4 Key Technical Flows 21
Flow 1 — Phone OTP Authentication 21
Flow 2 — Mobile Money Payment 21
Flow 3 — Property Listing Creation & Verification 22
7. Frontend Design System 22
7.1 Design Principles 23
7.2 Colour Palette 23
7.3 Typography 23
7.4 Core Screen Inventory 24
7.5 Internationalisation (i18n) 24
8. Security Design 25
8.1 Authentication Security 26
8.2 API Security 26
8.3 Payment Security 26
8.4 Data Privacy 26
8.5 Infrastructure Security 26
9. Third-Party Integrations 27
9.1 Campay Payment Integration Detail 28
10. Deployment & Infrastructure 29
10.1 Environment Strategy 30
10.2 CI/CD Pipeline (GitHub Actions) 30
10.3 Infrastructure Costs (Monthly Estimate — Early Stage) 30
11. Development Roadmap 30
Phase 1 — MVP Web Platform (Weeks 1–10) 31
Phase 2 — Transactions & Mobile App (Weeks 11–24) 31
Phase 3 — Growth Features (Month 7–12) 31
12. Open Questions & Pre-Build Decisions 32
13. Glossary 33

1. Product Vision & Scope

1.1 Vision Statement

RentCam is a bilingual (French/English) digital marketplace for residential and commercial property rentals in Cameroon. It replaces the fragmented, informal, word-of-mouth rental process with a transparent, verified, and fully digital platform — from property discovery to lease signing and rent payment — built specifically for the Cameroonian market's infrastructure constraints, payment ecosystem, and cultural context.

1.2 Mission

To make finding, renting, and managing property in Cameroon as simple, safe, and affordable as possible — for tenants, landlords, and agents alike.

1.3 Product Goals

Goal	Success Metric	Timeline
Aggregate rental inventory	10,000 verified listings in Yaoundé + Douala	Month 6
Drive tenant adoption	50,000 registered tenants	Month 12
Monetise supply side	1,000 paid agent/landlord accounts	Month 12
Enable digital payments	2,000 transactions/month processed on-platform	Month 18
Establish trust	>4.2/5 avg platform trust score in surveys	Month 12

1.4 Scope — What RentCam IS

- A property listing and search marketplace
- A tenant–landlord communication and inquiry platform
- A digital lease generation and management tool
- A rent payment processing platform (via mobile money)
- A reputation and review system for tenants, landlords, and agents
- A property management dashboard for landlords and agents

1.5 Scope — What RentCam is NOT

- Not a real-estate agency (no brokerage license required; marketplace model)
- Not a property development company
- Not a mortgage or home-buying platform (rental only, Phase 1)
- Not a general classifieds site (strictly property rentals)

1.6 Platform Surfaces

Surface	Technology	Priority	Users
Web Application (PWA)	Next.js — server-side rendered	Phase 1	All user types; primary for landlords/agents

Surface	Technology	Priority	Users
Android Mobile App	React Native	Phase 2	Tenants (primary mobile segment)
iOS Mobile App	React Native (shared codebase)	Phase 2	Expat/diaspora segment
Admin Dashboard	Next.js (internal)	Phase 1	RentCam operations team
Agent CRM Portal	Next.js (sub-app)	Phase 2	Real-estate agents

2. Functional Requirements

2.1 User Roles & Permissions

Role	Description	Key Capabilities
Guest	Unauthenticated visitor	Browse listings, view details, view map
Tenant	Registered renter	Search, save, inquire, sign lease, pay, review
Landlord	Property owner	List, manage units, screen tenants, receive payment, review
Agent	Licensed/informal RE agent	Multi-listing management, lead inbox, CRM, promoted listings
Admin	RentCam operations staff	Full CRUD, listing verification, dispute resolution, analytics
Super Admin	Technical/management	Platform config, user management, system settings

2.2 Feature Breakdown by Module

Module 1 — Authentication & Identity

Feature	Description	Phase
Phone OTP registration	SMS OTP via Africa's Talking (cheaper than Twilio for CM)	1
Phone OTP login	Passwordless; phone is primary identifier	1
Profile creation	Name, photo, role (tenant/landlord/agent), city	1
ID document upload	CNI scan upload for identity verification (manual review)	1
Selfie verification	Selfie + ID photo match for landlord verification badge	2
Social login	Google OAuth as optional supplement	2
Two-factor auth	Optional TOTP for admin accounts	2

Module 2 — Property Listings

Feature	Description	Phase
Create listing	Landlord/agent submits: type, address, price, description, photos	1
Photo upload	Min 3 photos required; max 20; client-side compression before upload	1
Geolocation tagging	GPS auto-detect or manual map pin; stored as lat/lng PostGIS point	1

Feature	Description	Phase
Listing status management	Available / Rented / Pending / Hidden / Under Review	1
Listing expiry	Auto-expire after 90 days; renewal reminder at day 80	1
Edit & update listing	Landlord can update price, description, availability	1
Duplicate detection	Flag potential duplicate listings (same address + price)	1
Admin verification queue	Admin reviews new listings; approves, requests changes, or rejects	1
Promoted/featured listings	Paid boost — appears at top of search results for 7/14/30 days	1
Virtual tour link	Optional YouTube/video URL field	2
Bulk listing upload	CSV import for agents with large portfolios	2

Module 3 — Search & Discovery

Feature	Description	Phase
Full-text search	Search by neighbourhood, city, description; PostgreSQL full-text	1
Filter: price range	Min/max monthly rent slider	1
Filter: property type	Chambre, Studio, Apartment (2BR/3BR+), Villa, Commercial	1
Filter: bedrooms	1, 2, 3, 4+	1
Filter: city/neighbourhood	Dropdown + free text; seeded neighbourhood list	1
Filter: furnished	Furnished / Unfurnished / Semi-furnished	1
Map view	Google Maps embed showing all matching listings as pins	1
Sort options	Price (asc/desc), Newest, Most relevant	1
Saved searches	Tenant saves a filter set; gets push/SMS alerts on new matches	2
Price heatmap	Neighbourhood average rent visualisation on map	3
AI-powered recommendations	"Listings you may like" based on browsing history	3

Module 4 — Inquiries & Communication

Feature	Description	Phase
Inquiry form	Tenant submits message + viewing request from listing page	1
WhatsApp CTA	"Contact on WhatsApp" button with pre-filled message template	1
In-app messaging	Thread-based chat between tenant and landlord/agent per listing	2
Viewing scheduler	Tenant proposes viewing times; landlord confirms/declines	2
Automated reminders	SMS/push reminders 24h before scheduled viewing	2
Read receipts	Shows if inquiry has been read	2
Message templates	Pre-written responses for common landlord replies	2

Module 5 — Lease Management

Feature	Description	Phase
Digital lease generator	Bilingual (FR/EN) standardised lease form; filled online	2
Lease template library	Residential standard, furnished, commercial, short-term templates	2
E-signature	Both parties sign digitally via OTP confirmation	2
PDF export	Signed lease downloadable as PDF	2
Lease storage	Encrypted lease archive per tenant and per landlord	2
Lease renewal workflow	Alert 30 days before expiry; renewal or termination flow	3
Termination notice	Digital notice-of-termination with timestamp and reason	3

Module 6 — Payments

Feature	Description	Phase
MTN Mobile Money collection	Via Campay or PayDunya API; push USSD prompt to tenant	2
Orange Money collection	Same API aggregator; both operators covered	2
Advance rent payment	Tenant pays 1–3 months advance through platform into escrow	2
Caution/deposit management	Deposit held in escrow; released to landlord on lease start	2

Feature	Description	Phase
Recurring rent payment	Monthly rent reminder + one-click payment via saved mobile money	3
Payment receipts	Auto-generated PDF receipt for every transaction	2
Landlord payout	Automated payout to landlord mobile money number after funds clear	2
Refund workflow	Deposit refund flow at end of tenancy; admin arbitration if disputed	3
Transaction history	Full ledger per user: credits, debits, pending, completed	2
Commission collection	Platform fee auto-deducted before landlord payout	2

Module 7 — Reviews & Trust

Feature	Description	Phase
Tenant reviews landlord	After lease ends; 1–5 stars + text; per-property	2
Landlord reviews tenant	After lease ends; private score affects tenant credibility score	2
Agent rating	Clients rate agent after successful match	2
Verified badge	Displayed on listings where landlord completed ID verification	1
Scam reporting	Any user can flag a listing; auto-hidden if 3+ flags; admin review	1
Review moderation	Admin can remove reviews that violate guidelines	2
Trust score	Composite score for tenants (payment history, reviews, verified ID)	3

Module 8 — Admin & Operations

Feature	Description	Phase
Listing verification queue	Admin reviews new/flagged listings with approve/reject/request-edit	1
User management	Ban, verify, impersonate (for support), role assignment	1
Dispute resolution	Structured dispute workflow between tenant and landlord	2
Analytics dashboard	Listings, users, searches, inquiries, payment volumes	1
Content moderation	Photo moderation queue; auto-detect inappropriate images (Phase 3)	1

Feature	Description	Phase
Notification management	Create and send bulk SMS/push to segments	2
Audit logs	Full action log for all admin operations	1

3. Non-Functional Requirements

Category	Requirement	Target
Performance	Page load time (web, 3G connection)	< 3 seconds
Performance	API response time (p95)	< 400ms
Performance	Image load (compressed thumbnails)	< 1 second
Availability	Platform uptime	> 99.5% monthly
Scalability	Concurrent active users	10,000+
Scalability	Listings in database	500,000+
Security	Data at rest	AES-256 encrypted
Security	Data in transit	HTTPS / TLS 1.3 only
Security	Payment data	Never stored on platform; tokenised via payment provider
Privacy	User data handling	Compliant with Cameroon Law No. 2010/012
Accessibility	Language support	French (primary) + English toggle
Accessibility	Low-bandwidth optimisation	Progressive image loading; offline-capable PWA
Mobile	Android minimum version	Android 6.0 (API 23)
Mobile	iOS minimum version	iOS 13
SEO	Web listing pages	SSR with structured data (schema.org/Apartment)

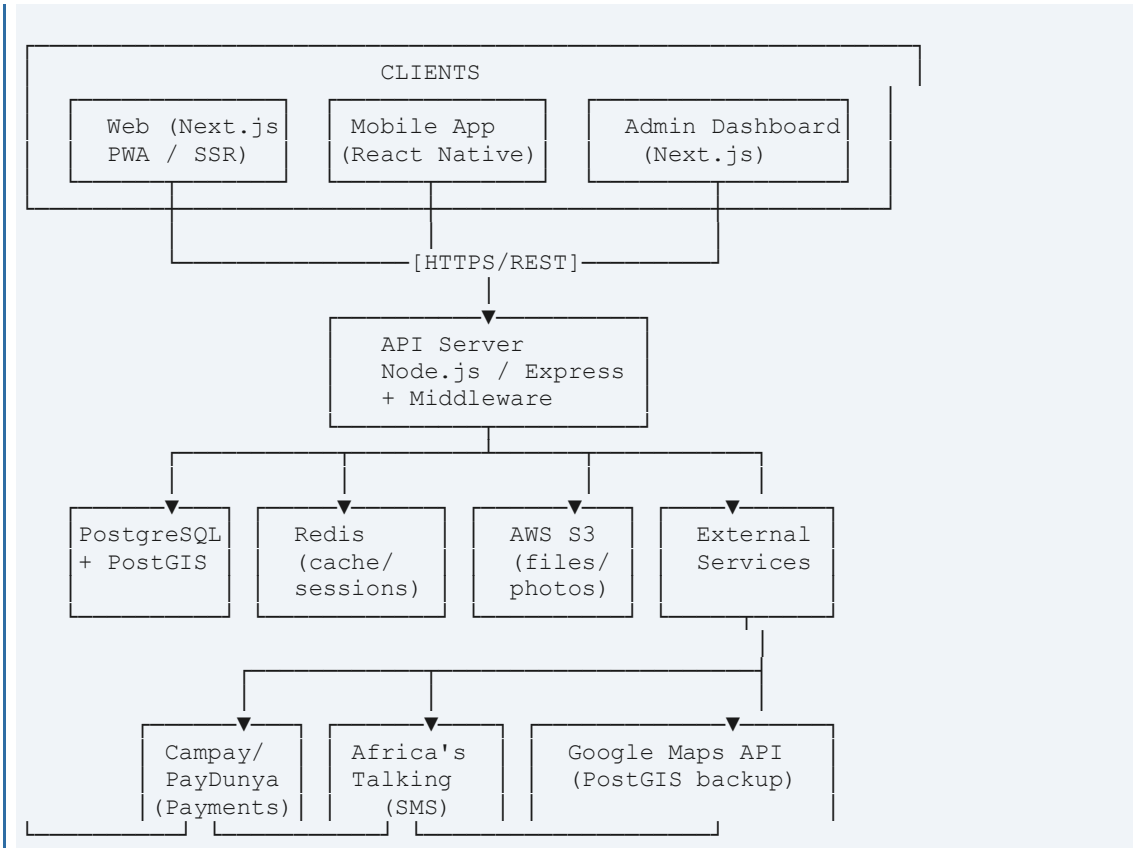
4. System Architecture

4.1 Architecture Overview

RentCam follows a monorepo, API-first architecture. A single Node.js/Express REST API serves both the Next.js web frontend and the React Native mobile app. The system is designed for a small team — pragmatic simplicity over microservices — while maintaining clear separation of concerns to allow scaling specific components as load increases.

4.2 High-Level Architecture Diagram

[Diagram — described below in text; implement in draw.io or Figma for visual representation]



4.3 Technology Stack — Detailed

Layer	Technology	Rationale
Monorepo tooling	Turborepo	Manage web + mobile + api + shared packages in one repo
Web framework	Next.js 14 (App Router)	SSR for SEO; API routes for BFF pattern; React ecosystem
Mobile framework	React Native 0.74 + Expo	Shared JS logic with web; Expo simplifies build pipeline

Layer	Technology	Rationale
API framework	Node.js 20 + Express 5	Team is JS-only; large ecosystem; async/await native
API validation	Zod	Schema validation + TypeScript inference from one definition
Database	PostgreSQL 16 + PostGIS	Relational integrity; geo queries; JSONB for flexible attrs
ORM	Drizzle ORM	Lightweight, type-safe, SQL-first; no magic; fast with PG
Cache / Sessions	Redis 7 (Upstash)	OTP storage, session tokens, rate limiting, search cache
File storage	Cloudflare R2	S3-compatible; free egress; cheaper than S3 for images
Image processing	Sharp (Node.js)	Resize, compress, convert to WebP on upload server-side
CDN / Edge	Cloudflare	African PoPs; image optimisation; DDoS; SSL termination
Maps	Google Maps JS API	Geocoding, place autocomplete, map embed
Payments	Campay API	Aggregates MTN MoMo + Orange Money; Cameroon-native; XAF
SMS / OTP	Africa's Talking	CM local SMS rates; OTP flow; bulk SMS for notifications
Email	Resend	Transactional email (receipts, alerts) — modern REST API
PDF generation	Puppeteer or @react-pdf/renderer	Lease PDF and receipt generation server-side
Auth	Custom JWT + Redis	Phone OTP → JWT access token + refresh token in Redis
Deployment — Web	Vercel	Zero-config Next.js deploy; automatic preview URLs
Deployment — API	Railway.app	Node.js container; managed PostgreSQL + Redis add-ons
Deployment — Mobile	Expo EAS Build	Cloud build + OTA updates without App Store resubmission
Monitoring	Sentry + Vercel Analytics	Error tracking; web vitals; API latency
CI/CD	GitHub Actions	Test → lint → build → deploy pipeline on push to main

4.4 API Architecture

4.4.1 API Design Principles

- RESTful resource-based endpoints — no GraphQL (team size, added complexity)

- Versioned routes: all endpoints prefixed /api/v1/
- JSON:API-compatible response envelope for consistent error handling
- Pagination: cursor-based for listings (stable under insertion); offset for admin tables
- Rate limiting: Redis sliding window; 100 req/min for authenticated, 20 for guest
- All timestamps in UTC ISO 8601; client handles timezone conversion

4.4.2 Response Envelope

```
// Success
{ "success": true, "data": { ... }, "meta": { "page": 1, "total": 243 } }

// Error { "success": false, "error": { "code": "LISTING_NOT_FOUND",
"message": "...", "field": "id" } }
```

4.4.3 Core API Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/auth/request-otp	Send OTP to phone number	None
POST	/api/v1/auth/verify-otp	Verify OTP; returns JWT + refresh token	None
POST	/api/v1/auth/refresh	Refresh access token	Refresh token
DELETE	/api/v1/auth/logout	Invalidate session	JWT
GET	/api/v1/listings	Search + filter listings (paginated)	Optional
POST	/api/v1/listings	Create new listing	Landlord/Agent
GET	/api/v1/listings/:id	Get listing details	Optional
PATCH	/api/v1/listings/:id	Update listing	Owner
DELETE	/api/v1/listings/:id	Soft-delete listing	Owner/Admin
POST	/api/v1/listings/:id/photos	Upload photos (multipart)	Owner
POST	/api/v1/listings/:id/inquiries	Submit tenant inquiry	Tenant
GET	/api/v1/listings/:id/inquiries	Get inquiries for listing	Owner
GET	/api/v1/users/me	Get authenticated user profile	JWT
PATCH	/api/v1/users/me	Update profile	JWT
POST	/api/v1/users/me/documents	Upload ID document	JWT
GET	/api/v1/users/:id/reviews	Get reviews for a user	Optional
POST	/api/v1/leases	Create draft lease	Landlord
GET	/api/v1/leases/:id	Get lease details	Parties
POST	/api/v1/leases/:id/sign	Sign lease (OTP confirmation)	Parties
POST	/api/v1/payments/initiate	Initiate mobile money payment	Tenant
POST	/api/v1/payments/callback	Campay webhook (server-to-server)	Campay HMAC

Method	Endpoint	Description	Auth
GET	/api/v1/payments/:id/status	Check payment status	JWT
POST	/api/v1/reviews	Submit review	JWT
POST	/api/v1/listings/:id/report	Report a listing	JWT
GET	/api/v1/admin/listings/queue	Verification queue	Admin
PATCH	/api/v1/admin/listings/:id/verify	Approve/reject listing	Admin

5. Database Design

5.1 Database Principles

- PostgreSQL 16 with PostGIS extension for geospatial property queries
- UUID primary keys on all tables (avoids enumerable IDs, safer for API exposure)
- Soft deletes (deleted_at timestamp) on listings, users, and leases — never hard delete
- All monetary amounts stored as INTEGER in XAF centimes (avoid floating point)
- JSONB columns for flexible, schema-less attributes (property features, metadata)
- Full-text search via PostgreSQL tsvector columns with GIN indexes
- All tables include created_at and updated_at; managed by trigger

5.2 Entity Relationship Overview

```
users ──< listings (landlord_id)
users ──< inquiries (tenant_id)
listings ──< inquiries
listings ──< listing_photos
listings ──< listing_amenities
users + listings ──< leases (tenant_id, property_id)
leases ──< payments
users ──< reviews (reviewer_id, reviewee_id)
listings ──< listing_reports
users ──< saved_listings
users ──< saved_searches users ──< notifications
```

5.3 Full Schema Definition

Table: users

```
CREATE TABLE users (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone           VARCHAR(20) UNIQUE NOT NULL,
  full_name       VARCHAR(200),
  email           VARCHAR(255) UNIQUE,
  role            user_role NOT NULL DEFAULT 'tenant',
                  -- ENUM: tenant | landlord | agent | admin | super_admin
  avatar_url      TEXT,
  bio             TEXT,
  city            VARCHAR(100),
  id_document_url TEXT,
  is_verified     BOOLEAN NOT NULL DEFAULT FALSE,
  verified_at     TIMESTAMPTZ,
  is_banned       BOOLEAN NOT NULL DEFAULT FALSE,
  ban_reason      TEXT,
  trust_score     SMALLINT DEFAULT 50 CHECK (trust_score BETWEEN 0 AND 100),
  preferred_lang  CHAR(2) NOT NULL DEFAULT 'fr',  -- 'fr' | 'en'
  last_seen_at    TIMESTAMPTZ,
  deleted_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_users_phone ON users(phone); CREATE INDEX idx_users_role
ON users(role);
```

Table: properties

```
CREATE TABLE properties (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  landlord_id      UUID NOT NULL REFERENCES users(id),
  agent_id         UUID REFERENCES users(id),  -- NULL if landlord lists
directly
  title            VARCHAR(300) NOT NULL,
  description      TEXT,
  property_type    property_type_enum NOT NULL,
                   -- ENUM: chambre|studio|apartment|villa|commercial|land
  status           listing_status NOT NULL DEFAULT 'pending_review',
                   -- ENUM:
pending_review|available|rented|reserved|hidden|rejected
  bedrooms         SMALLINT,
  bathrooms        SMALLINT,
  area_sqm         NUMERIC(8,2),
  floor            SMALLINT,
  total_floors     SMALLINT,
  furnished        furnished_enum NOT NULL DEFAULT 'unfurnished',
                   -- ENUM: furnished|semi_furnished|unfurnished
  monthly_rent     INTEGER NOT NULL,  -- XAF (not centimes; no subunit
needed)
  advance_months   SMALLINT NOT NULL DEFAULT 3,
  caution_months   SMALLINT NOT NULL DEFAULT 1,
  agency_fee_months SMALLINT DEFAULT 1,
  utilities        TEXT[],  -- ['water','electricity','internet']
  -- Location
  address_raw      TEXT NOT NULL,
  city             VARCHAR(100) NOT NULL,
  neighbourhood    VARCHAR(200),
  location         GEOGRAPHY(POINT, 4326),  -- PostGIS
  -- Extras
  amenities        TEXT[],  -- ['parking','security','generator','borehole']
  rules            TEXT[],  -- ['no_pets','no_smoking','couples_ok']
  available_from   DATE,
  virtual_tour_url TEXT,
  metadata         JSONB DEFAULT '{}',
  -- Search
  search_vector    TSVECTOR,  -- populated by trigger
  -- Promotion
  is_featured      BOOLEAN DEFAULT FALSE,
  featured_until   TIMESTAMPTZ,
  -- Stats
  view_count       INTEGER NOT NULL DEFAULT 0,
  inquiry_count    INTEGER NOT NULL DEFAULT 0,
  -- Admin
  rejection_reason TEXT,
  reviewed_by      UUID REFERENCES users(id),
  reviewed_at      TIMESTAMPTZ,
  expires_at       TIMESTAMPTZ DEFAULT NOW() + INTERVAL '90 days',
  deleted_at       TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Geospatial index for proximity search
CREATE INDEX idx_properties_location ON properties USING GIST(location);
-- Full-text search index
CREATE INDEX idx_properties_search   ON properties USING GIN(search_vector);
-- Common filter indexes
CREATE INDEX idx_properties_status    ON properties(status);
```

```
CREATE INDEX idx_properties_city      ON properties(city);
CREATE INDEX idx_properties_type      ON properties(property_type);
CREATE INDEX idx_properties_rent      ON properties(monthly_rent); CREATE
INDEX idx_properties_landlord  ON properties(landlord_id);
```

Table: listing_photos

```
CREATE TABLE listing_photos (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id  UUID NOT NULL REFERENCES properties(id) ON DELETE CASCADE,
  url          TEXT NOT NULL,        -- Cloudflare R2 object URL
  thumbnail_url TEXT,               -- 400×300 WebP thumbnail
  position     SMALLINT NOT NULL DEFAULT 0,  -- display order
  is_cover     BOOLEAN NOT NULL DEFAULT FALSE,
  width        INTEGER,
  height       INTEGER,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
); CREATE INDEX idx_photos_property ON listing_photos(property_id);
```

Table: inquiries

```
CREATE TABLE inquiries (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id     UUID NOT NULL REFERENCES properties(id),
  tenant_id       UUID NOT NULL REFERENCES users(id),
  message         TEXT,
  desired_move_in DATE,
  duration_months SMALLINT,
  status          inquiry_status NOT NULL DEFAULT 'sent',
                  -- ENUM: sent|read|replied|viewing_scheduled|closed
  viewing_date    TIMESTAMPTZ,
  landlord_reply  TEXT,
  replied_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_inquiries_property ON inquiries(property_id); CREATE INDEX
idx_inquiries_tenant   ON inquiries(tenant_id);
```

Table: leases

```
CREATE TABLE leases (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id         UUID NOT NULL REFERENCES properties(id),
  tenant_id           UUID NOT NULL REFERENCES users(id),
  landlord_id         UUID NOT NULL REFERENCES users(id),
  inquiry_id          UUID REFERENCES inquiries(id),
  lease_number        VARCHAR(50) UNIQUE,  -- human-readable ref e.g. RC-
2026-00142
  status              lease_status NOT NULL DEFAULT 'draft',
                      -- ENUM:
draft|pending_tenant_sign|pending_landlord_sign|active|expired|terminated
  -- Terms
  start_date          DATE NOT NULL,
  end_date            DATE,  -- NULL for open-ended
  duration_months     SMALLINT,
  monthly_rent        INTEGER NOT NULL,  -- XAF
  advance_months      SMALLINT NOT NULL,
  advance_amount      INTEGER NOT NULL,  -- XAF
```

```
  caution_amount      INTEGER NOT NULL,  -- XAF (deposit)
  caution_status      caution_status DEFAULT 'pending',
                      -- ENUM: pending|held|released|disputed
  -- Signatures
  tenant_signed_at    TIMESTAMPTZ,
  landlord_signed_at  TIMESTAMPTZ,
  tenant_sign_otp     VARCHAR(10),  -- cleared after signing
  landlord_sign_otp   VARCHAR(10),
  -- Document
  pdf_url             TEXT,
  template_id         VARCHAR(50),
  lease_data          JSONB,  -- full lease content snapshot
  -- Termination
  terminated_at       TIMESTAMPTZ,
  termination_reason  TEXT,
  terminated_by       UUID REFERENCES users(id),
  deleted_at          TIMESTAMPTZ,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_leases_property  ON leases(property_id);
CREATE INDEX idx_leases_tenant    ON leases(tenant_id);
CREATE INDEX idx_leases_landlord  ON leases(landlord_id); CREATE INDEX
idx_leases_status    ON leases(status);
```

Table: payments

```
CREATE TABLE payments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lease_id          UUID REFERENCES leases(id),
  payer_id          UUID NOT NULL REFERENCES users(id),
  payee_id          UUID NOT NULL REFERENCES users(id),
  amount            INTEGER NOT NULL,  -- XAF
  platform_fee      INTEGER NOT NULL DEFAULT 0,  -- XAF deducted
  net_amount        INTEGER NOT NULL,  -- amount - platform_fee
  payment_type      payment_type_enum NOT NULL,
                    -- ENUM:
advance_rent|caution|monthly_rent|platform_fee|refund
  method            payment_method NOT NULL,
                    -- ENUM: mtn_momo|orange_money|cash|bank_transfer
  status            payment_status NOT NULL DEFAULT 'pending',
                    -- ENUM: pending|processing|completed|failed|refunded
  provider_ref      VARCHAR(100),  -- Campay transaction ID
  provider_status   VARCHAR(50),
  payer_phone       VARCHAR(20),
  receipt_url       TEXT,
  notes             TEXT,
  paid_at           TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_payments_lease  ON payments(lease_id);
CREATE INDEX idx_payments_payer  ON payments(payer_id); CREATE INDEX
idx_payments_status ON payments(status);
```

Table: reviews

```
CREATE TABLE reviews (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reviewer_id  UUID NOT NULL REFERENCES users(id),
  reviewee_id  UUID NOT NULL REFERENCES users(id),
  property_id  UUID REFERENCES properties(id),
```

```
  lease_id     UUID REFERENCES leases(id),
  direction    review_direction NOT NULL,
               -- ENUM: tenant_to_landlord | landlord_to_tenant |
tenant_to_agent
  rating       SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment      TEXT,
  is_public    BOOLEAN NOT NULL DEFAULT TRUE,
  is_flagged   BOOLEAN NOT NULL DEFAULT FALSE,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_reviews_once_per_lease   ON reviews(reviewer_id,
lease_id, direction);  -- one review per party per lease
```

Table: otp_tokens

```
-- Stored in Redis (TTL 10 minutes), not PostgreSQL
-- Key:   otp:{phone}
-- Value: { code: '482917', attempts: 0, createdAt: ISO }
-- TTL:   600 seconds -- Max attempts: 3 (then block for 1 hour)
```

Additional Supporting Tables

Table	Key Columns	Purpose
saved_listings	user_id, property_id, created_at	Tenant favourites / shortlist
saved_searches	user_id, filters JSONB, alert_enabled, last_alerted_at	Saved filter sets with push alerts
listing_reports	reporter_id, property_id, reason, details, status, resolved_by	Scam/fraud flagging queue
notifications	user_id, type, title, body, channel (push/sms/email), read_at, metadata JSONB	Notification inbox
audit_logs	actor_id, action, entity_type, entity_id, changes JSONB, ip_address, created_at	Admin action audit trail
promotions	property_id, type (featured/boosted), starts_at, ends_at, payment_id	Paid listing promotions
neighbourhoods	id, city, name, slug, lat, lng, avg_rent_1br, avg_rent_2br, updated_at	Neighbourhood master data + price averages

6. Application Layer Design

6.1 Monorepo Structure

```
rentcam/
├── apps/
│   ├── web/              # Next.js 14 (App Router)
│   ├── mobile/           # React Native + Expo
│   └── admin/            # Next.js internal dashboard
├── packages/
│   ├── api/              # Express API server
│   ├── db/               # Drizzle schema + migrations
│   ├── ui/               # Shared React components
│   ├── types/            # Shared TypeScript types
│   └── config/           # ESLint, Tailwind, tsconfig presets
├── turbo.json
├── package.json └── .env.example
```

6.2 API Server Structure (packages/api)

```
packages/api/src/
├── index.ts              # Entry point; mounts app
├── app.ts                # Express app; middleware setup
├── routes/
│   ├── auth.routes.ts
│   ├── listings.routes.ts
│   ├── users.routes.ts
│   ├── inquiries.routes.ts
│   ├── leases.routes.ts
│   ├── payments.routes.ts
│   ├── reviews.routes.ts
│   └── admin.routes.ts
├── controllers/          # Route handlers (thin — call services)
├── services/             # Business logic (listings, payments, leases...)
├── middleware/
│   ├── auth.middleware.ts    # JWT verification
│   ├── role.middleware.ts    # Role-based guard
│   ├── rateLimit.middleware.ts
│   ├── upload.middleware.ts  # Multer + Sharp
│   └── validate.middleware.ts # Zod schema validation
├── lib/
│   ├── redis.ts
│   ├── storage.ts        # R2 client
│   ├── campay.ts         # Payment provider
│   ├── sms.ts            # Africa's Talking
│   └── pdf.ts            # Lease PDF generator └── utils/
```

6.3 Next.js Web App Structure (apps/web)

```
apps/web/src/
├── app/
│   ├── (public)/
│   │   ├── page.tsx              # Homepage / search landing
│   │   ├── listings/
│   │   │   ├── page.tsx          # Search results
```

```
│   │   │   └── [id]/page.tsx     # Listing detail (SSR + schema.org)
│   │   ├── auth/
│   │   │   ├── login/page.tsx
│   │   │   └── register/page.tsx
│   │   └── neighbourhoods/[slug]/page.tsx
│   ├── (tenant)/
│   │   ├── dashboard/page.tsx
│   │   ├── saved/page.tsx
│   │   ├── inquiries/page.tsx
│   │   └── leases/[id]/page.tsx
│   ├── (landlord)/
│   │   ├── dashboard/page.tsx
│   │   ├── listings/
│   │   │   ├── new/page.tsx
│   │   │   └── [id]/edit/page.tsx
│   │   └── tenants/page.tsx
│   └── (agent)/
│       ├── dashboard/page.tsx
│       └── portfolio/page.tsx
├── components/
│   ├── search/          # SearchBar, FilterPanel, MapView, ListingCard
│   ├── listing/         # PhotoGallery, ListingDetail, InquiryForm
│   ├── dashboard/       # StatCard, OccupancyChart, PaymentHistory
│   ├── auth/            # PhoneInput, OTPInput
│   └── shared/          # Button, Input, Modal, Toast, Skeleton...
├── hooks/               # useSearch, useGeolocation, useAuth...
├── lib/                 # API client, auth helpers └── i18n/                #
fr.json, en.json translation files
```

6.4 Key Technical Flows

Flow 1 — Phone OTP Authentication

Step	Actor	Action	Notes
1	Client	POST /auth/request-otp { phone }	Phone format validated (Cameroon +237)
2	API	Generate 6-digit OTP	Crypto.randomInt(100000, 999999)
3	API	Store in Redis: otp:{phone} → {code, attempts:0}	TTL: 600s
4	API	Send SMS via Africa's Talking	Message: 'RentCam code: 482917'
5	Client	User enters OTP; POST /auth/verify-otp { phone, code }	
6	API	Fetch from Redis; compare codes; increment attempts	Block after 3 failures
7	API	Delete OTP from Redis	Prevent reuse
8	API	Upsert user record (create if new)	is_new flag in response
9	API	Issue JWT (15min) + refresh token (30 days in Redis)	
10	Client	Store tokens; redirect to dashboard or onboarding	

Flow 2 — Mobile Money Payment

Step	Actor	Action	Notes
1	Tenant	Clicks 'Pay Advance Rent' on lease page	
2	Client	POST /payments/initiate { leaseId, method: 'mtn_momo', phone }	
3	API	Create payment record (status: pending)	
4	API	Call Campay: collect({ amount, from, description })	Campay pushes USSD to tenant phone
5	Tenant	Approves payment on phone (USSD prompt)	~30-60 seconds
6	Campay	POST /payments/callback (webhook)	HMAC signature verified
7	API	Update payment status → completed	
8	API	If caution: update lease.caution_status → held	
9	API	If advance: update lease status → active	
10	API	Schedule landlord payout (T+1 business day)	Minus platform fee
11	API	Send receipt SMS + email to both parties	
12	Client	Polling /payments/:id/status shows 'completed'	Or push notification

Flow 3 — Property Listing Creation & Verification

Step	Actor	Action
1	Landlord	Fills listing form: type, address, price, description
2	Client	Geolocation API captures lat/lng; Google Places fills address fields
3	Client	Photos selected; client-side Sharp compression to ≤500KB each
4	Client	POST /listings (multipart); photos uploaded to Cloudflare R2
5	API	Create property record (status: pending_review); store photo URLs
6	API	Trigger: update search_vector tsvector field
7	API	Notify admin via dashboard + optional SMS: 'New listing to review'
8	Admin	Reviews listing in admin queue: photos, price, address, geolocation
9	Admin	PATCH /admin/listings/:id/verify { status: 'available' }
10	API	Update property status → available; set expires_at
11	API	Send SMS to landlord: 'Your listing is now live'
12	Tenant	Listing appears in search results

7. Frontend Design System

7.1 Design Principles

- Mobile-first: all layouts designed for 375px width, scaled up to desktop
- Data-light: lazy loading images, skeleton screens, optimistic UI updates
- Bilingual: all strings externalised to fr.json / en.json; language toggle persistent
- Accessibility: WCAG 2.1 AA target; proper ARIA labels; keyboard navigable
- Low-literacy friendly: icons alongside all text labels; clear visual hierarchy

7.2 Colour Palette

Token	Hex	Usage
Primary 600	#1A6FA8	CTAs, links, active states
Primary 100	#DBEAFE	Backgrounds, selected states
Success 600	#16A34A	Available badge, payment success
Warning 500	#F59E0B	Pending states, advance notice
Danger 600	#DC2626	Error states, rented badge, reports
Neutral 900	#111827	Body text
Neutral 600	#4B5563	Secondary text
Neutral 200	#E5E7EB	Borders, dividers
Neutral 50	#F9FAFB	Page backgrounds
White	#FFFFFF	Card backgrounds

7.3 Typography

Role	Font	Size / Weight
Display (hero)	Inter	36–48px / 700
Heading 1	Inter	28px / 700
Heading 2	Inter	22px / 600
Heading 3	Inter	18px / 600
Body	Inter	16px / 400
Body small	Inter	14px / 400
Label / caption	Inter	12px / 500
Monospace (prices)	Inter Mono	16px / 600

ℹ Inter is web-safe, free (Google Fonts), and renders clearly on low-res Android screens. Fallback: system-ui, sans-serif.

7.4 Core Screen Inventory

Screen	Route	Key Components
Home / Search	/	Hero search bar, city selector, featured listings, recent listings
Search Results	/listings	Filter panel, listing cards grid, map toggle, sort controls, pagination
Listing Detail	/listings/:id	Photo gallery, price + info, map, CTA buttons, contact form, reviews
Map View	/listings?view=map	Full-screen map, listing pins, hover card, filter overlay
Auth — Phone Entry	/auth/login	Phone number input (+237 prefix), country flag
Auth — OTP Verify	/auth/verify	6-digit OTP input, resend timer (60s)
Register / Onboarding	/auth/register	Name, role selection (tenant/landlord/agent), city
Tenant Dashboard	/dashboard	Active lease, saved listings, recent inquiries, notifications
Landlord Dashboard	/landlord/dashboard	Occupancy summary, payment status, listing health, inquiry inbox
Create Listing	/landlord/listings/new	Multi-step form: details → location → photos → preview → submit
Lease Detail	/leases/:id	Lease terms, parties, payment schedule, sign CTA, PDF download
Payment Flow	/pay/:leaseId	Amount breakdown, MoMo selector (MTN/Orange), USSD instructions
Admin Queue	/admin/listings/queue	Table of pending listings, preview pane, approve/reject/request-edit
Admin Dashboard	/admin	KPI cards, charts, recent activity, system health

7.5 Internationalisation (i18n)

All user-facing strings are managed via next-intl. Language is stored in a cookie and applied server-side for SSR.

```
// i18n/fr.json (excerpt)
{
  "search": {
    "placeholder": "Quartier, ville ou adresse...",
    "button": "Rechercher",
    "noResults": "Aucun logement trouvé. Essayez d'élargir votre recherche."
  },
  "listing": {
    "bedrooms": "{n} chambre(s)",
    "available": "Disponible",
```

```
    "rented": "Loué",
    "perMonth": "{amount} FCFA / mois"
  } }
```

8. Security Design

8.1 Authentication Security

- JWT access tokens: 15-minute expiry; HS256 signed with 256-bit secret
- Refresh tokens: opaque 256-bit random hex; stored in Redis with 30-day TTL
- OTP brute force protection: 3 attempts max; 1-hour block after failure; stored in Redis
- Phone number normalisation: all stored in E.164 format (+237XXXXXXXXX)
- No passwords stored on platform — phone OTP only

8.2 API Security

- All endpoints over HTTPS/TLS 1.3; Cloudflare handles termination
- CORS: whitelist only known origins (rentcam.cm, admin.rentcam.cm)
- Helmet.js: sets security headers (CSP, HSTS, X-Frame-Options, etc.)
- Rate limiting: 100 req/min authenticated, 20 req/min guest (per IP + per user)
- SQL injection: Drizzle ORM uses parameterised queries exclusively; no raw string interpolation
- Input validation: every endpoint has a Zod schema; unvalidated input never reaches DB
- File upload security: MIME type validation; file size cap 5MB; magic byte check

8.3 Payment Security

- Campay webhook verification: HMAC-SHA256 signature on every callback
- No card data stored; all payment data handled by Campay
- Idempotency: payment initiation uses idempotency keys to prevent double-charging
- Escrow model: advance rent never paid directly to landlord until lease is active

8.4 Data Privacy

- Tenant ID documents encrypted at rest in R2 using customer-managed AES-256 keys
- Phone numbers masked in public API responses (e.g. +237 6XX XXX 456)
- GDPR-inspired data deletion: user can request account deletion; soft-delete + 30-day purge
- Audit logs retained for 2 years; personal data anonymised after account deletion

8.5 Infrastructure Security

- Environment variables in Railway secrets / Vercel env; never committed to git
- Least-privilege DB user: API connects as rentcam_app (no DDL permissions)
- VPC isolation: DB not publicly accessible; API connects via private network
- Automated dependency scanning via GitHub Dependabot
- OWASP Top 10 review before each major release

9. Third-Party Integrations

Service	Provider	Purpose	Integration Method
Mobile Money	Campay (aggregator)	MTN MoMo + Orange Money collection & payout	REST API; webhook callbacks
SMS / OTP	Africa's Talking	OTP delivery, payment alerts, lease notifications	REST API
Maps / Geocoding	Google Maps Platform	Address autocomplete, map embed, geocoding	JS API (client) + Geocoding API (server)
File Storage	Cloudflare R2	Property photos, lease PDFs, ID documents	S3-compatible SDK
Email	Resend	Transactional: receipts, lease copies, alerts	REST API
Error Monitoring	Sentry	Frontend + API error capture & alerting	SDK
Analytics	Vercel Analytics + PostHog	Web vitals, funnel analysis, feature flags	SDK
Push Notifications	Firebase Cloud Messaging	Mobile push for inquiries, payment confirmations	React Native SDK

9.1 Campay Payment Integration Detail

```
// packages/api/src/lib/campay.ts
import axios from 'axios';

const campay = axios.create({
  baseURL: process.env.CAMPAY_BASE_URL,  // https://demo.campay.net/api or
live
  headers: { Authorization: `Token ${process.env.CAMPAY_TOKEN}` }
});

export async function initiateCollection(params: {
  amount: number;      // XAF
  currency: 'XAF';
  from: string;        // payer phone e.g. 237671234567
  description: string;
  external_reference: string;  // our payment.id
}) {
  const { data } = await campay.post('/collect/', params);
  return data; // { reference, ussd_code, status: 'PENDING' }
}

export function verifyWebhookSignature(body: string, signature: string):
boolean {
  const expected = crypto
    .createHmac('sha256', process.env.CAMPAY_WEBHOOK_SECRET!)
    .update(body).digest('hex');
  return crypto.timingSafeEqual(
    Buffer.from(signature), Buffer.from(expected)
  ); }
```

10. Deployment & Infrastructure

10.1 Environment Strategy

Environment	Purpose	URL Pattern
Development	Local development; hot reload	localhost:3000 (web), localhost:4000 (api)
Staging	QA, UAT, integration testing	staging.rentcam.cm
Production	Live platform	rentcam.cm, api.rentcam.cm

10.2 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml (simplified)
on: push to main

jobs:
  test:    Run ESLint + TypeScript check + Jest unit tests
  build:   Turbo build all apps; check for build errors
  migrate: Run Drizzle migrations against staging DB
  deploy:  Vercel deploy (web + admin); Railway deploy (api)   notify:
Slack notification on success/failure
```

10.3 Infrastructure Costs (Monthly Estimate — Early Stage)

Service	Plan	Est. Monthly Cost (USD)
Vercel (web + admin)	Pro	$20
Railway (API + PostgreSQL + Redis)	Starter	$25–50
Cloudflare R2 (storage)	Pay-as-you-go	$5–15
Google Maps Platform	Pay-as-you-go	$10–30
Africa's Talking SMS	Pay-per-message	$10–30
Campay	% of transactions	Included in take rate
Sentry	Team plan	$26
Resend (email)	Pro	$20
Domain + SSL	Cloudflare Registrar	$15/year
Total (MVP running)		~$130–200/month

11. Development Roadmap

Phase 1 — MVP Web Platform (Weeks 1–10)

Week(s)	Sprint Goal	Deliverables
1–2	Project Setup	Monorepo, DB schema, auth (OTP), CI/CD pipeline, staging env
3–4	Listings Core	Create listing, photo upload (R2), listing detail page (SSR)
5–6	Search Engine	Full-text search, filters, map view (Google Maps), listing cards
7	Inquiry System	Inquiry form, WhatsApp CTA, landlord inquiry inbox
8	Admin Tools	Verification queue, user management, basic analytics dashboard
9	Trust Features	Verified badges, scam reporting, neighbourhood data
10	Polish + Launch Prep	i18n (FR/EN), mobile responsiveness, performance, SEO, beta testing

Phase 2 — Transactions & Mobile App (Weeks 11–24)

Week(s)	Sprint Goal	Deliverables
11–13	Digital Leases	Lease generator, bilingual template, OTP e-signature, PDF export
14–16	Payments	Campay integration (MTN + Orange), escrow, payout, receipt generation
17–18	Reviews	Tenant/landlord mutual review system, post-lease flow
19–21	React Native App (Android)	Auth, search, listing detail, inquiry, push notifications
22–23	Landlord Dashboard	Occupancy, payment history, tenant management, maintenance log
24	iOS App + Performance	iOS build, app store submissions, load testing

Phase 3 — Growth Features (Month 7–12)

Feature	Description
Saved searches + alerts	SMS/push when new listing matches tenant's saved filter
Recurring rent payments	Monthly rent payment link + auto-reminder flow
Agent CRM module	Portfolio management, lead pipeline, bulk listing tools
Short-term / furnished rentals	Separate category; daily/weekly pricing; availability calendar

Feature	Description
Price heatmaps	Neighbourhood average rent visualisation
Rent financing partnerships	Integration with local MFIs for advance rent loans
Commercial property segment	Office, retail, warehouse listings
Diaspora dashboard	Multi-currency display, international time zones, diaspora-specific UX

12. Open Questions & Pre-Build Decisions

#	Question	Options	Recommendation
1	Payment aggregator choice	Campay vs PayDunya vs direct MTN/Orange APIs	Start with Campay — fastest to integrate, both operators, XAF native
2	Listing moderation model	Manual review vs automated + manual for flagged	Manual Phase 1 (builds trust); automate Phase 2 with photo AI
3	Commission structure	Fixed fee vs % of transaction vs hybrid	% model (2–3%) scales; add fixed fee for promoted listings
4	Agent onboarding	Self-serve vs white-glove vs field agents	Field agents in Yaoundé + Douala for first 200 agents
5	Company registration	SA vs SARL vs foreign entity	SARL in Cameroon (Yaoundé) — required to hold payment funds
6	Search engine	PostgreSQL FTS vs Meilisearch	PG FTS for MVP; migrate to Meilisearch if search latency >200ms at scale
7	Escrow legality	Platform holding funds vs immediate transfer	Consult BEAC/COBAC on e-money licensing requirements before Phase 2
8	Offline support	Full offline vs cache-only vs none	Service Worker cache for search results page (PWA); full offline Phase 3

13. Glossary

Term	Definition
Caution	Security deposit (typically 1–2 months rent) held against damage/unpaid rent
Avance	Advance rent payment (typically 3–6 months) required upfront in Cameroon
Gérant	Property manager (often informal) managing units on behalf of a landlord
CNI	Carte Nationale d'Identité — Cameroonian national ID card
FCFA / XAF	CFA Franc — Central African currency; 1 EUR ≈ 655 XAF
Crieur	Informal town-crier who announces available properties in neighbourhoods
PostGIS	PostgreSQL extension for geospatial data; enables radius searches by lat/lng
OTP	One-Time Password — 6-digit SMS code used for passwordless authentication
MoMo	Mobile Money — digital wallet service (MTN MoMo is Cameroon's largest)
SSR	Server-Side Rendering — page HTML generated on server for SEO and fast load
PWA	Progressive Web App — web app installable on phone home screen; offline capable
Drizzle ORM	TypeScript-first SQL query builder and schema manager for PostgreSQL
Campay	Cameroon-based payment aggregator integrating MTN MoMo and Orange Money
Escrow	Platform holds payment funds temporarily before releasing to landlord